Smart Canteen Billing System

Project Statement | Python Console Application

1. Introduction

School, college and office canteens serve many customers in a short time, and slow or inaccurate billing causes long queues and stock problems. Keeping track of what has been sold, what is running out and how much money has come in through cash, UPI and cards is hard to do on paper. The **Smart Canteen Billing System** is a command-line application written in pure Python that handles all of this in one program.

It combines billing, live stock tracking, three payment methods, a prepaid smart-card wallet, an admin panel and a sales report. All data is saved in JSON files, so menu, stock, wallet balances and sales history are still there after the program is closed and reopened. It needs only Python 3.7 or later and no extra packages.

2. Objectives

- Speed up billing with a cart-based order screen and automatic calculation of discount and GST.
- Keep stock accurate by reducing it after every sale and warning when items run low.
- Support the payment methods canteens actually use: cash, UPI and a prepaid card.
- Give the canteen owner simple tools to manage the menu and review sales.
- Save data permanently and keep a receipt for every bill.

3. Key Features

Menu and live stock	Seven starter items (Samosa, Veg Sandwich, Masala Dosa, Veg Thali, Tea, Coffee, Cold Drink) with price and stock. Items are marked "low!" or "sold out" automatically.
Cart-based billing	Add items, remove items, view the cart, check out or cancel. Quantity is limited to the stock available, so overselling is not possible.
Discount and GST	10% discount when the bill is above Rs.300, then 5% GST on the discounted amount.
Cash payment	Checks the amount received and shows the change to return.
UPI payment (simulated)	Accepts a UPI ID containing @, sends a simulated request and asks the cashier to confirm receipt.
Smart Card wallet	Register a card, top it up, check the balance and pay from it. Payment is refused if the balance is too low.
Receipts	Each bill is printed on screen and saved as its own file (bill_0001.txt, bill_0002.txt and so on).
Admin panel	PIN-protected. Add items, change prices, restock, remove items, view low-stock items and open the sales report.

Sales report	Bills and sales for all time and for today, the top five selling items and the money collected by payment mode.
Data persistence	Menu, wallets and sales are stored in JSON files inside the canteen_data folder. Broken or missing files are handled safely.

4. How It Works

The main menu lets the user start a new order, view the menu, open the smart-card menu, enter the admin panel or exit. A typical sale follows these steps:

- **Build the cart:** the cashier picks item IDs and quantities, and the program checks them against current stock.
- **Calculate the bill:** subtotal, then the discount if the subtotal is above Rs.300, then GST, then the grand total.
- **Take payment:** cash, UPI or smart card. If payment fails, the cart is kept so the cashier can try again.
- **Finish the sale:** stock is reduced, the sale is recorded, the receipt is printed and saved, and a low-stock alert is shown if needed.

Settings such as GST percentage, discount limit, low-stock level and admin PIN are constants at the top of the file, and the code is split into small functions for files, input, billing, payments, reports and menus, so each part is easy to change.

5. Technologies and Concepts Used

- **Language:** Python 3.7+ using only the standard library (json, os and datetime).
- **Concepts:** dictionaries, lists, functions, loops, exception handling (try/except), JSON file storage, f-string formatting, sorting and input validation.
- **Design:** separate helper functions, configuration constants and a safe start-up that creates the data folders and default menu on the first run.

6. Conclusion

The Smart Canteen Billing System goes beyond a simple bill calculator. It brings ordering, stock control, multiple payment modes, a prepaid wallet and sales reporting into one easy-to-use program, and it keeps its data between sessions. This reduces billing errors, shortens queues and gives the canteen owner a clear view of what is selling and what needs restocking.

Possible future improvements include a customer-facing display, barcode or QR scanning, real UPI integration, hashed admin PINs and user accounts, daily and monthly report exports, and a graphical or web interface. As a project it also shows how a complete real-world application can be built in clean, readable Python without any outside libraries.